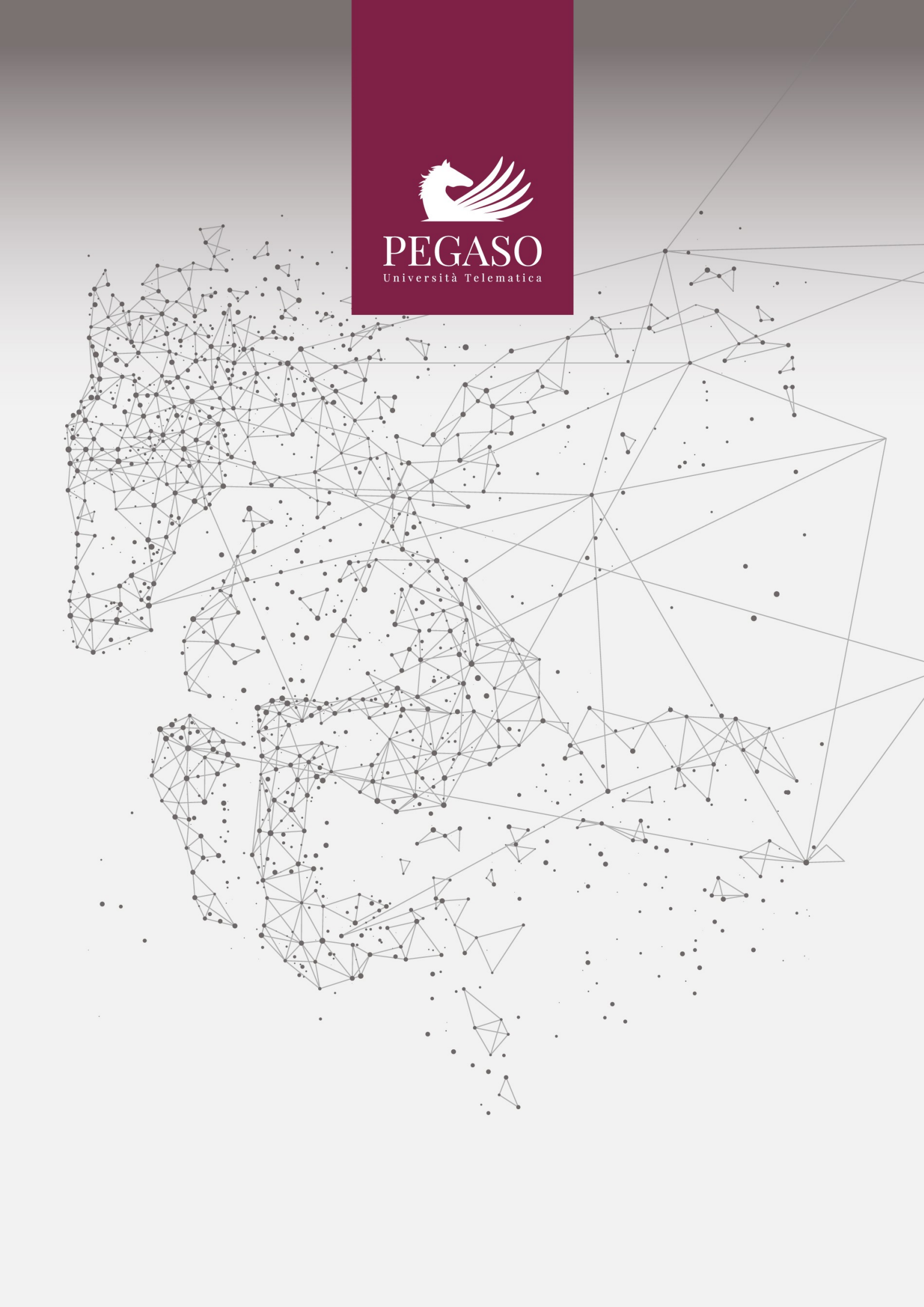

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. AJAX 3
2. XHR E FETCH API 5
3. AJAX – JQUERY 8
BIBLIOGRAFIA 13
SITOGRAFIA 14

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Ajax

Le chiamate **sincrone** e **asincrone** sono un concetto fondamentale per comprendere il funzionamento di AJAX.

Una chiamata sincrona, o bloccante, è un tipo di chiamata in cui il thread principale di esecuzione del programma viene bloccato fino a quando la chiamata non viene completata. In altre parole, quando si effettua una chiamata sincrona, il programma non può eseguire alcuna altra operazione fino a quando la chiamata non è stata completata. Questo comportamento è spesso indesiderato in applicazioni web, poiché può causare il blocco dell'interfaccia utente e una cattiva esperienza utente.

Al contrario, una chiamata asincrona, o non bloccante, è un tipo di chiamata in cui il thread principale di esecuzione del programma non viene bloccato durante l'esecuzione della chiamata. In altre parole, quando si effettua una chiamata asincrona, il programma può continuare a eseguire altre operazioni, senza dover attendere il completamento della chiamata. Questo comportamento è molto utile in applicazioni web, poiché consente di creare interfacce utente fluide e reattive.

AJAX (Asynchronous JavaScript and XML) è una tecnica di programmazione che consente di eseguire richieste HTTP asincrone utilizzando JavaScript: l'obiettivo di AJAX è quello di creare interfacce utente dinamiche e reattive che non richiedono il ricaricamento completo della pagina web. Quando dunque viene eseguita una richiesta AJAX, il programma continua ad eseguire altre operazioni, senza dover attendere il completamento della richiesta; quando la risposta del server è disponibile, viene gestita in modo asincrono, senza interrompere l'esecuzione del programma.

Per implementare AJAX, è necessario utilizzare JavaScript per eseguire una richiesta HTTP asincrona al server; **XMLHttpRequest** e **Fetch API** sono dei metodi "nativi" perchè direttamente implementati nel browser; tuttavia, ci sono anche altre librerie che consentono di semplificare tali chiamate, come ad es. jQuery, Axios, SuperAgent ecc.

Il dettaglio del flusso di lavoro di una richiesta AJAX può essere descritto nel modo seguente:

- **Evento nella pagina web**: l'utente interagisce con la pagina web, ad esempio cliccando su un pulsante o caricando la pagina.
- **Creazione di un oggetto XMLHttpRequest**: quando si verifica un evento, JavaScript crea un nuovo oggetto XMLHttpRequest. Questo oggetto ci consente di eseguire richieste HTTP asincrone al server.
- **Invio della richiesta al server**: l'oggetto XMLHttpRequest invia una richiesta HTTP al server. La richiesta può essere una richiesta GET, POST, PUT, DELETE o un'altra.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- **Elaborazione della richiesta**: il server elabora la richiesta e genera una risposta.
- **Invio della risposta al client**: il server invia la risposta al client, che la riceve come una stringa di dati.
- **Lettura della risposta**: JavaScript legge la risposta dal server e la converte in un formato utilizzabile.
- **Aggiornamento della pagina web**: infine, JavaScript utilizza i dati ricevuti dal server per aggiornare l'interfaccia utente della pagina web. Ad esempio, può aggiornare il contenuto degli elementi HTML, creare nuovi elementi HTML o aggiornare lo stile CSS degli elementi esistenti.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. XHR e fetch API

Le due principali API utilizzate per effettuare richieste AJAX nei browser sono **XMLHttpRequest** (**XHR**) e **Fetch API**. XHR è stata la prima API implementata per effettuare richieste AJAX, mentre Fetch API è stata introdotta più di recente con l'obiettivo di semplificare l'API XHR e fornire funzionalità aggiuntive.

Fetch API è considerata generalmente più semplice e intuitiva da utilizzare rispetto a XHR: tale interfaccia web integrata ci consente di eseguire richieste HTTP e di gestire la risposta del server in modo semplice e intuitivo. La Fetch API è stata introdotta in JavaScript ES6 (ECMAScript 2015) ed è disponibile in tutti i browser moderni, inclusi Chrome, Firefox, Edge e Safari.

La Fetch API è progettata per essere più semplice e flessibile rispetto all'oggetto XMLHttpRequest (XHR) utilizzato tradizionalmente per le richieste http; in particolare, la Fetch API utilizza una sintassi più semplice e intuitiva basata su **Promises** e ci consente di utilizzare i dati della risposta in modo più diretto e conveniente.

Le **Promises** sono un costrutto introdotto in JavaScript ES6 (ECMAScript 2015) che ci consente di gestire il flusso di esecuzione asincrona in modo più efficiente e leggibile. Una Promise rappresenta un'operazione asincrona che potrebbe non essere ancora completata e ci consente di gestire il risultato dell'operazione una volta che è disponibile.

In pratica, le Promises ci permettono di gestire il flusso di esecuzione di una serie di operazioni asincrone in modo più intuitivo e leggibile, senza dover annidare callback all'interno di callback. Possiamo utilizzare le Promises per eseguire operazioni asincrone come le richieste HTTP, le animazioni, le operazioni di elaborazione dati e altre.

Ecco un esempio di come effettuare una richiesta AJAX per ottenere un file JSON utilizzando sia l'API XHR che Fetch API:

```
// XHR
const xhr = new XMLHttpRequest();
xhr.open('GET', 'https://example.com/data.json');
xhr.onload = () => {
  if (xhr.status === 200) {
    const data = JSON.parse(xhr.responseText);
    console.log(data);
  }
};
xhr.send();

// FETCH API
fetch('https://example.com/data.json')
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
  .then(response => {
    if (response.ok) {
      return response.json();
    }
    throw new Error('Network response was not ok.');
})
.then(data => console.log(data))
.catch(error => console.error(error));
```

Entrambi gli esempi fanno una richiesta HTTP GET all'URL 'https://example.com/data.json', ottengono la risposta JSON e la stampa nella console. Tuttavia, ci sono alcune differenze tra le due implementazioni:

- Con XHR:
    - è necessario creare un'istanza dell'oggetto XMLHttpRequest, aprire una connessione verso l'URL specificato, impostare una funzione di callback per gestire la risposta e inviare la richiesta
    - la risposta viene gestita nella funzione di callback impostata tramite onload
    - la gestione degli errori viene effettuata nella funzione di callback impostata tramite onerror
- Con Fetch API:
    - la funzione fetch restituisce una Promise che rappresenta la risposta HTTP alla richiesta effettuata; quando la Promise viene risolta, il metodo then viene chiamato e riceve in input un oggetto Response, che rappresenta la risposta HTTP.
    - il primo blocco then controlla se la risposta HTTP ha avuto successo (response.ok restituisce true se la risposta ha uno stato HTTP 2xx). In caso di successo, viene chiamato il metodo json() sulla risposta Response per ottenere il corpo della risposta in formato JSON, che viene restituito come una nuova Promise.
    - il secondo blocco then riceve in input il risultato della Promise restituita dal metodo json(), ovvero il corpo della risposta in formato JSON. In questo caso, la funzione semplicemente stampa il corpo della risposta nella console.
    - se si verifica un errore durante la richiesta HTTP o durante il parsing del corpo della risposta in formato JSON, viene lanciata una eccezione e la Promise viene rifiutata. In questo caso, il blocco catch viene chiamato e riceve in input l'errore, che viene stampato nella console.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

In sintesi, sia l'XHR che la Fetch API sono strumenti potenti per implementare AJAX e gestire le richieste HTTP asincrone. La scelta tra XHR e Fetch dipende dalle preferenze del programmatore e dalle esigenze del progetto. In generale, la Fetch API è più moderna e semplificata, ma l'XHR è più consolidato e offre una maggiore compatibilità con i browser più vecchi.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Ajax – jQuery

jQuery fornisce un metodo chiamato $.ajax() che semplifica la creazione di richieste AJAX personalizzate, offrendo un'interfaccia più semplice rispetto all'utilizzo diretto di XHR o Fetch API.

Ecco un esempio di utilizzo del metodo $.ajax() per effettuare una richiesta GET al server e ricevere una risposta in formato JSON:

```
$.ajax({
  url: 'https://example.com/data.json',
  method: 'GET',
  dataType: 'json',
  success: function(data) {
    console.log(data);
  },
  error: function(jqXHR, textStatus, errorThrown) {
    console.error(textStatus, errorThrown);
  }
});
```

In questo esempio, il metodo $.ajax() viene chiamato con un oggetto di configurazione che specifica l'URL della risorsa da richiedere, il metodo HTTP da utilizzare (GET in questo caso), il tipo di dati attesi in risposta (JSON in questo caso) e due funzioni di callback per gestire la risposta: success e error.

La funzione success viene chiamata quando la richiesta HTTP ha avuto successo e restituisce il corpo della risposta nel formato specificato dalla proprietà dataType (in questo caso, JSON). La funzione error viene chiamata quando la richiesta HTTP ha fallito e riceve in input tre parametri: l'oggetto jqXHR che rappresenta la richiesta AJAX, lo stato del server e un eventuale messaggio di errore.

jQuery offre molte altre funzionalità per semplificare l'utilizzo di AJAX. Ecco alcuni esempi:

$.get()

Questo metodo semplifica la creazione di richieste HTTP GET al server, fornendo un'interfaccia ancora più semplice rispetto al metodo $.ajax(). Il metodo $.get() richiede solo l'URL della risorsa da richiedere e una funzione di callback per gestire la risposta, senza dover specificare il metodo HTTP, il tipo di dati attesi o altre opzioni avanzate.

```
$.get('https://example.com/data.json', function(data) {
  console.log(data);
});
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

$.post()

Questo metodo semplifica la creazione di richieste HTTP POST al server, fornendo un'interfaccia simile a quella di $.get(). Il metodo $.post() richiede solo l'URL della risorsa da richiedere, i dati da inviare al server e una funzione di callback per gestire la risposta

```
$.post('https://example.com/data', { name: 'John', age: 30 },
function(data) {
  console.log(data);
});
```

$.getJSON()

Questo metodo semplifica la creazione di richieste HTTP GET per ottenere risposte in formato JSON, fornendo un'interfaccia ancora più semplice rispetto a $.get(). Il metodo $.getJSON() richiede solo l'URL della risorsa da richiedere e una funzione di callback per gestire il corpo della risposta JSON

```
$.getJSON('https://example.com/data.json', function(data) {
  console.log(data);
});
```

$.ajaxSetup()

Questo metodo consente di impostare opzioni di default per tutte le richieste AJAX effettuate con jQuery, semplificando la configurazione globale della libreria. Ad esempio, è possibile impostare il tipo di dati attesi in risposta o l'URL base per tutte le richieste AJAX

```
$.ajaxSetup({
  dataType: 'json',
  beforeSend: function(xhr) {
    xhr.setRequestHeader('Authorization', 'Bearer ' + token);
  }
});
```

In generale, l'utilizzo di jQuery semplifica notevolmente la creazione di richieste AJAX personalizzate, fornendo una sintassi più semplice e chiara rispetto alle API XHR e Fetch. Tuttavia, va tenuto presente che l'utilizzo di una libreria esterna come jQuery comporta un overhead aggiuntivo e può non essere necessario in tutti i progetti.

Consideriamo il seguente esempio: viene creata una pagina HTML con un bottone e un div vuoto. Quando l'utente clicca sul bottone, viene effettuata una richiesta AJAX GET all'URL 'https://jsonplaceholder.typicode.com/todos/1', che restituisce un oggetto JSON di esempio.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Quando la risposta viene ricevuta con successo, la funzione di callback success utilizza il metodo html di jQuery per aggiungere i dati ricevuti nel div #data-container. In questo esempio, i dati vengono semplicemente convertiti in una stringa JSON utilizzando il metodo JSON.stringify.

In caso di errore durante la richiesta AJAX, viene chiamata la funzione di callback error, che stampa l'errore nella console del browser.

In questo modo, è possibile effettuare richieste AJAX tramite jQuery e visualizzare i risultati nella pagina stessa, senza dover effettuare un nuovo caricamento della pagina.

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>AJAX con jQuery</title>
  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
  <script>
    $(document).ready(function() {
      $('#btn-get-data').click(function() {
        $.ajax({
          url: 'https://jsonplaceholder.typicode.com/todos/1',
          method: 'GET',
          dataType: 'json',
          success: function(data) {
            $('#data-container').html('<p>'+JSON.stringify(data)+'</p>');
          },
          error: function(jqXHR, textStatus, errorThrown) {
            console.error(textStatus, errorThrown);
          }
        });
      });
    });
  </script>
</head>
<body>
  <h1>AJAX con jQuery</h1>
  <button id="btn-get-data">Ottieni dati</button>
  <div id="data-container"></div>
</body>
</html>
```

Il sito web 'https://jsonplaceholder.typicode.com/' è un servizio web di test che fornisce dati JSON di esempio. In particolare, il servizio prevede diverse risorse JSON che rappresentano dati di esempio per la gestione di post, commenti, attività da fare e utenti.

Il servizio è utilizzato principalmente per scopi di sviluppo e di testing di applicazioni web che utilizzano dati JSON in input o in output. Ad esempio, è possibile utilizzare il servizio per testare la connessione e l'interazione con servizi web esterni, come API RESTful, o per apprendere come gestire i dati JSON in JavaScript.

Per accedere alle diverse risorse JSON del servizio, è possibile utilizzare gli endpoint forniti dal sito:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- /posts → 100 posts
- /comments → 500 comments
- /albums → 100 albums
- /photos → 5000 photos
- /todos → 200 todos
- /users 10 users

Ad esempio, l'endpoint '/todos' restituisce la lista completa delle attività da fare, mentre l'endpoint '/users' restituisce la lista degli utenti del servizio; un esempio di restituzione è https://jsonplaceholder.typicode.com/posts:


```
[
  {
    "userId": 1,
    "id": 1,
    "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
    "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
  },
  {
    "userId": 1,
    "id": 2,
    "title": "qui est esse",
    "body": "est rerum tempore vitae\nsequi sint nihil reprehenderit dolor beatae ea dolores neque\nfugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis\nqui aperiam non debitis possimus qui neque nisi nulla"
  },
…
```


Se voglio un post specifico basta operare nella seguente maniera:

https://jsonplaceholder.typicode.com/posts/10


```
{
  "userId": 1,
  "id": 10,
  "title": "optio molestias id quia eum",
  "body": "quo et expedita modi cum officia vel magni\ndoloribus qui repudiandae\nvero nisi sit\nquos veniam quod sed accusamus veritatis error"
}
```


La scelta di jQuery per chiamate AJAX rispetto all'utilizzo di Fetch API risulta comunque una scelta del tutto personale.

È vero che la Fetch API ha molti vantaggi rispetto a jQuery.ajax(), ad esempio:

- È inclusa di default in tutti i moderni browser web;

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- Supporta i Promise, che semplificano la gestione delle richieste asincrone e consentono di scrivere codice più leggibile e meno propenso ad errori;
- Supporta nativamente i dati in formato JSON, senza la necessità di doverli parsare manualmente come avviene con jQuery.ajax();
- Consente di inviare richieste con diversi metodi HTTP, non solo GET e POST come in jQuery.ajax();
- Consente di inviare richieste con body, che possono contenere diversi tipi di dati, tra cui anche JSON;
- Consente di definire custom headers per la richiesta.
- Tuttavia, ci sono alcuni casi in cui può essere utile utilizzare jQuery.ajax() invece della Fetch API:
- Se si utilizza jQuery in modo globale nell'applicazione e si desidera utilizzare le funzionalità AJAX fornite da jQuery senza dover importare ulteriori librerie;
- Se si desidera utilizzare una sintassi più familiare e semplificata per gestire le richieste AJAX, poiché la sintassi di jQuery.ajax() è più breve rispetto a quella della Fetch API;
- Se si deve gestire un vecchio browser che non supporta la Fetch API.

In generale, è sempre importante scegliere la tecnologia più adatta alle proprie esigenze e al contesto specifico, valutando i vantaggi e gli svantaggi di ciascuna opzione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- V. Roberto, M. Frailis, A. Gugliotta, P. Omero. Introduzione alle Tecnologie Web, McGraw-Hill, 2010.
- A. S. Tanembaum. Computer Networks (4 ed.). Prentice Hall (4 ed.), 2003.
- Marco Andreini, Patrizia Andronico, Arianna Del Soldato, Cristian Lucchesi, Maurizio Martinelli, Giuseppe Vasarelli. Manuale corso di Tecnologie Web Istituto di Informatica e Telematica del CNR, 2005.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Sitografia

- HTML 4.01 Specification: http://www.w3.org/TR/html401/
- HTML5: http://www.w3.org/TR/html5/
- Javascript: JavaScript Tutorial (w3schools.com)
- HTML: http://www.w3schools.com/html/default.asp
- CSS: https://www.w3schools.com/css/default.asp
- CSS: http://www.w3.org/TR/CSS/

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).